



AI Stock Flag Bot

Complete Project Documentation

Architecture · Every file · Every strategy · AI concepts · System design
Real-money transition guide · Optimization roadmap · Interview preparation

Built July 2026 · 100% free tier · Repo: github.com/teknoninja/stocks-trading-agent
Educational project — not financial advice

1. What Is This Project? (Plain Language)

This project is a **personal AI trading assistant**. When you browse any stock on TradingView, a chat panel appears beside the chart. It reads the stock's price history, runs **17 professional trading strategies** on it, and raises a flag: **BUY**, **SELL**, or **HOLD** — with a confidence score and plain-English reasons. You can ask it questions ("why this flag?", "where is the support?") and it answers like an analyst sitting next to you.

It also **practice-trades automatically**. With fake money (called "paper trading"), it buys and sells stocks from your watchlists on a schedule — US stocks through a broker simulator called Alpaca, Indian stocks through a simulator we built ourselves — even while your laptop is switched off, because the schedule runs on GitHub's computers in the cloud. Every decision is recorded and scored against what the market actually did, so over months you get an honest answer to the question: *"is this bot actually good?"*

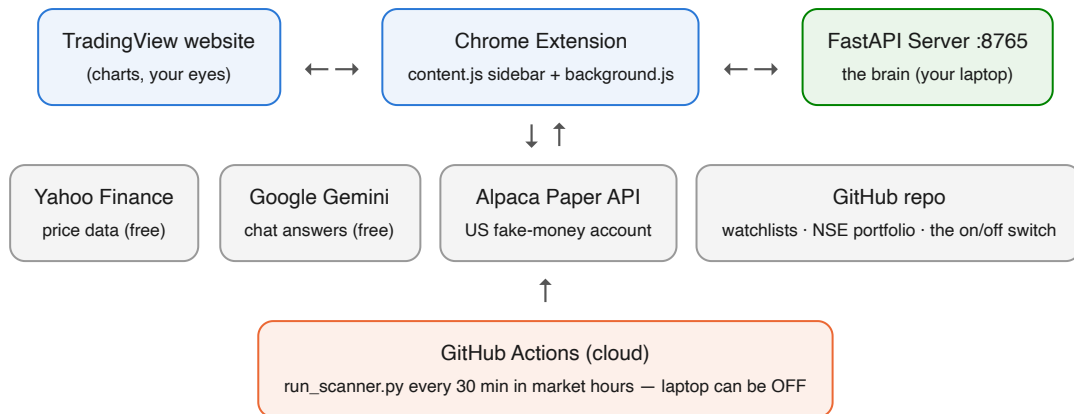
The five characters in the story

Character	Role (analogy)	Technology
The Engine	A committee of 17 experienced chart-readers who each vote buy/sell/neutral. Their weighted votes become the flag.	Pure Python math (pandas/numpy) — deterministic, no AI model
The Spokesperson	Explains the committee's evidence in plain language and answers your questions. Never invents its own trading opinion.	Google Gemini (free-tier LLM), grounded on the engine's output
The Window	The sidebar that follows you around TradingView showing the flag, buttons, and chat.	Chrome extension (Manifest V3) — content script + service worker
The Hands	Practice accounts that actually place the paper trades and remember your positions.	Alpaca paper API (US) + our own virtual broker (India)
The Night-Shift Worker	Wakes up every 30 minutes during market hours, checks the watchlists, trades, goes back to sleep. Works when you don't.	GitHub Actions scheduled workflow (cloud, free)

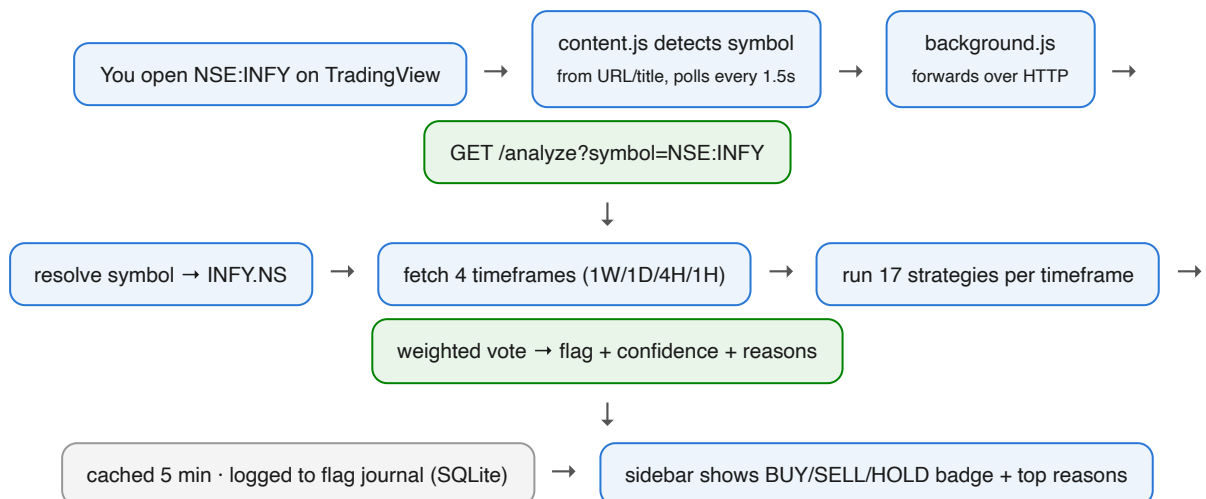
The single most important design decision: the buy/sell decision is made by **deterministic mathematics**, not by an AI language model. The AI (Gemini) only *explains*. Why? Math gives the same answer every time, can be backtested against history, and never "hallucinates" a price level. An LLM asked "should I buy?" gives different answers on different days and invents numbers. We use each tool for what it is good at — this split is the answer to the interview question "where did you use AI and where deliberately not?"

2. System Design

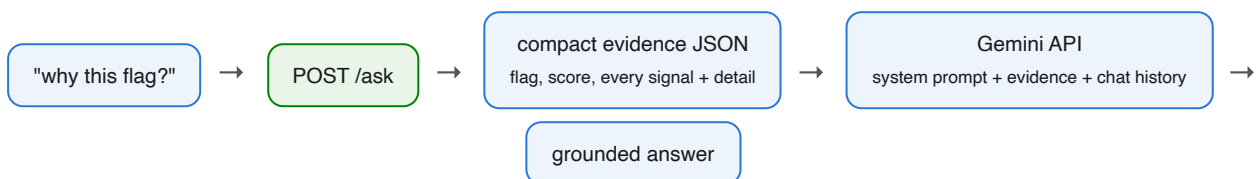
2.1 Bird's-eye architecture



2.2 Flow 1 — You open a stock (the flag flow)

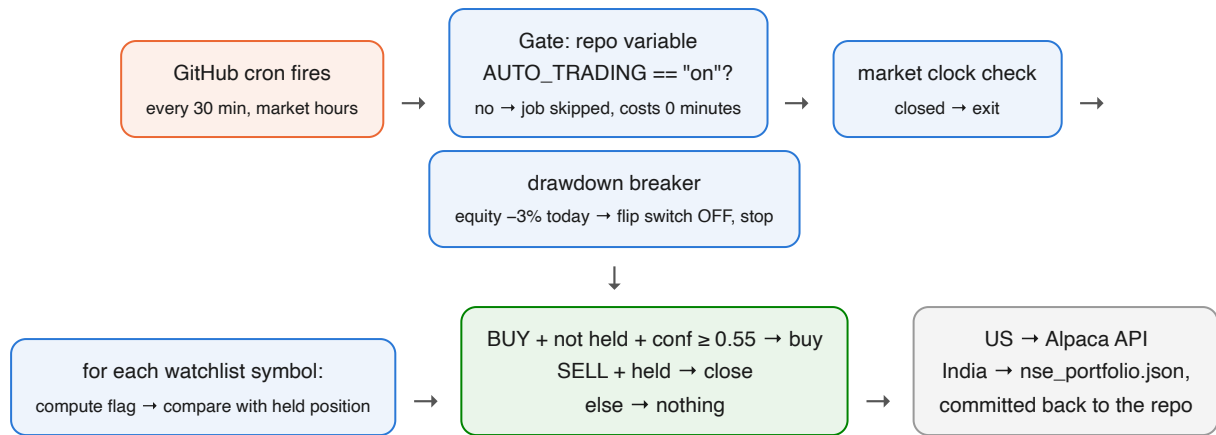


2.3 Flow 2 — You ask a question (the chat flow)



If Gemini is overloaded → try next model in ladder → if no API key at all → rule-based text summary. The bot never goes fully silent.

2.4 Flow 3 — Auto-trading while you sleep (the scanner flow)



2.5 Where does state live? (a key design table)

The scanner is **stateless** — it keeps no memory between runs. Every run re-derives its decisions from two sources of truth it can always reach. This is why the same code runs identically on your laptop or on a throwaway cloud machine.

Data	Lives where	Why there
US positions, orders, cash, P&L	Alpaca's servers	A real (simulated) brokerage account — survives everything
Indian virtual portfolio + equity history	<code>nse_portfolio.json</code> tracked inside the git repo	Cloud runs check out the repo, trade, commit the file back — "GitHub as a tiny database"
Watchlists (the scanner's entire universe)	<code>watchlist.txt</code> / <code>watchlist_in.txt</code> in the repo	Same reason; the ★ button edits them via GitHub's API
The ON/OFF switch	GitHub repo variable <code>AUTO_TRADING</code>	Must be readable by cloud runs when your laptop is off
Flag journal + outcomes	SQLite file <code>data/flag_journal.db</code> (local)	Only used for your local scoreboard — losing it loses no money state
Secrets (API keys)	<code>.envrc</code> / setx locally + GitHub encrypted Secrets	Never in code, never committed (gitignored)
Analysis results	In-memory cache, 5-min TTL	Pure derived data — safe to lose, cheap to recompute

3. Repository Tour — Every File Explained

3.1 Project root

File	What it is, in plain words
<code>run_tradingview_bot.py</code>	The main launcher. Starts the analysis server; optionally also opens TradingView in a Playwright-driven Chrome with the sidebar injected (legacy mode — the Chrome extension replaced this for daily use). <code>--no-browser</code> runs the server only.
<code>run_scanner.py</code>	One reconciliation pass of the auto-trader. <code>--dry-run</code> prints decisions without trading; <code>--market us in both</code> picks the market. This is the script GitHub Actions runs.
<code>run_backtest.py</code>	Replays the engine over years of history to measure whether the flags would have made money. Prints hit-rates per flag and per strategy; saves CSVs to <code>backtest_results/</code> .
<code>watchlist.txt</code> / <code>watchlist_in.txt</code>	Plain text lists of symbols the auto-scanner may trade (US / India). The scanner's <i>only</i> universe. Managed by the ★ button, the /watchlists page, or any text editor.
<code>nse_portfolio.json</code>	The Indian virtual account: cash, positions, trade log, equity history. Tracked in git on purpose (see 2.5).
<code>start_bot.bat</code> / <code>start_bot.command</code>	One-double-click launchers for Windows / macOS.
<code>requirements.txt</code> / <code>requirements-scanner.txt</code>	Full dependency list vs. the minimal set the cloud scanner needs (keeps CI fast).
<code>pyproject.toml</code> , <code>uv.lock</code>	Python project metadata and locked dependency versions.
<code>.envrc</code> / <code>.envrc.example</code>	Your secret keys (gitignored) and the template showing which variables exist.
<code>README.md</code> , <code>WINDOWS_SETUP.md</code>	Setup and usage docs.
<code>.github/workflows/paper-trader.yml</code>	The cloud schedule: two cron lines (US + NSE market hours), the AUTO_TRADING gate, secret injection, and a step that commits the NSE portfolio back to the repo.

3.2 `stocks_agent/` — the original AI agents package

File	Explanation
<code>tools.py</code>	18 data tools (company info, EPS trends, earnings, news, SEC filings, social sentiment, screeners, and our <code>get_technical_flag</code>). Each is a plain Python function wrapped with <code>function_tool</code> so LLM agents can call it — this is function calling / tool use , the core pattern of modern AI agents.
<code>simple_agent.py</code>	Stateless Q&A agent (OpenAI Agents SDK): every question starts fresh. Demonstrates the minimal agent loop: LLM picks tools → tools run → LLM writes the answer.
<code>conversation_agent.py</code>	Same, plus conversation memory and automatic ticker tracking across turns.
<code>structured_agent.py</code>	Returns both a narrative and a 30-field validated object (Pydantic structured output) — the pattern for making LLM output machine-readable and schema-guaranteed.
<code>free_agent.py</code>	The same agent running on free local models via Ollama (Qwen/Llama) — a hand-rolled tool-calling loop, useful to explain how agent frameworks work under the hood.
<code>__init__.py</code>	Exports everything; heavy LLM imports are wrapped in try/except so the lightweight cloud scanner can import the package without installing LLM SDKs.

3.3 `stocks_agent/technical/` — the decision engine

File	Explanation
<code>data.py</code>	All market data access. Maps TradingView symbols to Yahoo symbols (NSE:INFY → INFY.NS, BINANCE:BTCUSDT → BTC-USD), fetches weekly/daily/4H/1H candles, resamples 1H→4H, and handles corporate-proxy TLS plus Yahoo rate limits (chart API first, yfinance fallback, clear error when throttled).
<code>indicators.py</code>	The math library: SMA, EMA, RSI, MACD, ATR, ADX, Bollinger Bands, z-score, anchored VWAP, OBV — hand-written in pandas (no TA-Lib dependency).
<code>structure.py</code>	Finds swing highs/lows (fractals), classifies trend (higher-highs/higher-lows = up), detects BOS and CHOC breaks.
<code>zones.py</code>	Supply/demand zones, order blocks, liquidity sweeps — the "smart money" price-action concepts.
<code>volume_profile.py</code>	Volume-by-price histogram → VPOC and the 70% value area.
<code>divergence.py</code>	RSI and MACD divergence at confirmed swing points.
<code>patterns.py</code>	Harmonic patterns (Gartley/Bat/Butterfly/Crab via Fibonacci ratios), an Elliott-wave heuristic, Wyckoff accumulation/distribution (spring/upthrust).
<code>quant.py</code>	Systematic signals: z-score/Bollinger mean reversion with an ADX trend filter, EMA-stack + golden-cross trend following, Donchian breakout with volume confirmation, anchored-VWAP stance.
<code>options_signals.py</code>	Options positioning from the nearest expiry: put/call open-interest ratio and ATM implied volatility.
<code>scoring.py</code>	The voting machine: every signal has direction (−1/0/+1), strength (0–1), weight; the normalized weighted sum in [−1,1] becomes BUY (≥ +0.22) / SELL (≤ −0.22) / HOLD, plus a confidence blend.
<code>engine.py</code>	The conductor: fetch frames → run every strategy per timeframe → multi-timeframe alignment → options → aggregate. <code>analyze_ticker()</code> for live, <code>analyze_frames()</code> for the backtester.
<code>backtest.py</code>	Walk-forward replay: every 5 trading days, compute the flag using only data available on that day, then measure the next 5/10/20-day returns. No look-ahead by construction.

3.4 `tradingview_bot/` — the product layer

File	Explanation
<code>server.py</code>	The FastAPI app — every endpoint the sidebar and pages use: <code>/health</code> , <code>/analyze</code> , <code>/ask</code> (Gemini chat with fallback ladder), <code>/papertrade</code> (routes US→Alpaca, India→virtual broker; explicit buy/sell sides), <code>/autotrade</code> (the GitHub-variable toggle), <code>/portfolio</code> + <code>/portfolio/reset</code> , <code>/watchlist*</code> endpoints, and two HTML dashboards: <code>/performance</code> and <code>/watchlists</code> .
<code>alpaca.py</code>	Thin REST client for the Alpaca paper API: account, clock, positions, notional market orders (tagged <code>tvbot-</code>), close position, and free IEX daily bars used as a data fallback.
<code>scanner.py</code>	The autonomous trader: guardrails (market clock, confidence floor, max positions, drawdown circuit breaker) + the reconcile loop for US (Alpaca) and India (virtual broker), plus read/write of the <code>AUTO_TRADING</code> GitHub variable.
<code>virtual_broker.py</code>	Our own paper brokerage for NSE: JSON portfolio, whole-share fills at latest price (min-1-share fallback), P&L, equity snapshots for the chart, and GitHub contents-API sync.
<code>watchlists.py</code>	Watchlist logic: market classification of any symbol, read/write of both files, GitHub sync, toggle/status used by the ★ button.
<code>journal.py</code>	The accountability layer: SQLite log of every flag; an outcome updater that fetches what price did 5/10/20 trading days later; hit-rate aggregation; the scoreboard HTML.
<code>sidebar.js</code>	The injected sidebar UI (Playwright variant — same features as the extension).
<code>browser.py</code>	Playwright launcher for the legacy injected mode.

3.5 `chrome_extension/` — the daily driver

File	Explanation
<code>manifest.json</code>	Manifest V3 declaration: run <code>content.js</code> on <code>tradingview.com</code> pages, allow the background worker to call <code>http://127.0.0.1:8765</code> .
<code>content.js</code>	The sidebar itself: symbol detection (URL param, <code>/symbols/</code> path, chart title), flag card, ★ watch toggle, Buy/Sell buttons, Auto toggle, chat. Runs inside the TradingView page.
<code>background.js</code>	A tiny service worker that proxies all API calls. Content scripts on an https page can't reliably call a local http server (CSP + local-network rules); the extension background can, thanks to <code>host_permissions</code> . Messaging pattern: <code>content</code> → <code>chrome.runtime.sendMessage</code> → worker fetches → response back.

4. The 17 Trading Strategies, In Simple Language

Each strategy is a small, independent detector. Each emits a **signal**: a direction (bullish +1 / bearish -1 / neutral 0), a strength (0–1, "how clear is this?"), and a weight ("how much do we trust this kind of evidence?"). Higher timeframes count more (weekly $\times 1.3$, daily $\times 1.0$, 4-hour $\times 0.8$, 1-hour $\times 0.6$).

Group A — Price action & market structure

1. Market Structure (HH/HL vs LH/LL) weight 1.5

Idea: a healthy uptrend climbs like stairs — each peak (high) and each dip (low) is higher than the previous one. A downtrend is the mirror image.

How the code sees it: it finds "swing points" — a bar whose high is the highest of its 3 neighbours on each side is a swing high (a fractal). Comparing the last few swings tells us: higher-highs+higher-lows = uptrend, lower-highs+lower-lows = downtrend, mixed = range.

2. BOS & CHOC (Break of Structure / Change of Character) weight 1.2

Idea: when price closes beyond the previous stair-step *in the trend direction*, the trend is confirmed continuing (BOS). When it breaks the step *against* the trend, the trend's character may be changing (CHOC) — an early reversal alert, so CHOC gets more strength than BOS.

Code: compares recent closes against the last swing high/low; only breaks within the last 10 bars count as "news".

3. Supply & Demand Zones weight 1.0

Idea: when price suddenly rockets out of a quiet area, big institutions were probably filling large orders there. If price returns to that area, those players often defend it — so quiet-zone- before-rocket-up = "demand zone" (expect bounce), quiet-zone- before-drop = "supply zone".

Code: a departure $\geq 2 \times \text{ATR}$ within 3 bars marks the base candles before it as a zone; a zone dies ("mitigated") once price closes through its far side; the signal fires when current price re-enters a still-fresh zone. ATR (Average True Range) = the stock's typical bar size, our unit of "sudden".

4. Order Blocks weight 0.8

Idea: the last red candle before a big rally (or last green before a dump) often marks exactly where institutions loaded positions. Price returning to that candle's range often reacts.

Code: finds the most recent impulse, walks backwards to the last opposite-coloured candle, and signals when price is testing that candle's range.

5. Liquidity Sweeps (stop hunts) weight 0.9

Idea: many traders place stop-loss orders just beyond obvious swing points. Price sometimes spikes through those levels — triggering the stops, letting big players fill orders — then snaps back. A sweep *below* lows that closes back above is bullish (sellers exhausted), and vice versa.

Code: a bar whose wick pierces a prior swing level but whose close comes back inside it, within the last 8 bars.

Group B — Indicator confluence

6. Multi-Timeframe Alignment weight 1.5

Idea: a setup is far more reliable when the weekly, daily, and 4-hour pictures agree. Trading a 1-hour bounce against a weekly downtrend is swimming upstream.

Code: computes the structure trend on 1W, 1D, 4H independently; all agree = strong signal, majority = weak, split = neutral.

7 & 8. RSI / MACD Divergence weight 1.0 each

Idea: price makes a higher high, but momentum (RSI or MACD) makes a lower high — the engine is revving less on each push. Classic early-reversal warning. (RSI measures recent up-vs-down force 0–100; MACD compares a fast and a slow average of price.)

Code: compares indicator values *at the last two confirmed swing points* — not at arbitrary bars, which is the mistake naive implementations make.

9. Volume Profile (VPOC & Value Area) weight 0.8

Idea: instead of "when did trading happen" ask "at which *price* did trading happen". The price with the most historical volume (VPOC) acts like gravity; the band holding 70% of volume is the "value area". Price above value = market accepting higher prices (bullish); below = bearish.

Code: spreads each bar's volume across its high-low range into 40 price bins, finds the peak bin, expands around it to 70%.

10. Anchored VWAP weight 0.7

Idea: VWAP = the volume-weighted average price everyone paid since some anchor moment — the "fair price" of that whole move. Institutions benchmark against it. Price above the VWAP anchored at the last major swing = buyers in control since that swing.

Code: cumulative (typical price × volume) / cumulative volume from the anchor bar; neutral when price is within 0.3×ATR of it.

Group C — Pattern-based methods

11. Harmonic Patterns (Gartley, Bat, Butterfly, Crab) weight 0.6

Idea: five-point zigzags (X-A-B-C-D) whose legs relate by specific Fibonacci ratios (0.618, 0.786, 1.27...). When the ratios line up and point D completes, traders expect a reversal there.

Code: takes the last 5 alternating swings, computes AB/XA, BC/AB, AD/XA, and matches them against each pattern's ratio windows with tolerance. D being a low = bullish pattern.

12. Elliott Wave Heuristic weight 0.4 (lowest — most subjective)

Idea: trends often unfold in 5 pushes then correct in 3. By push number 5, momentum usually fades — "wave-5 exhaustion".

Code: counts consecutive higher-highs/higher-lows; a 3rd+ new high with a *weaker RSI* than the previous high = possible exhaustion (contrarian signal). Deliberately low weight because honest Elliott counting is subjective even among experts.

13. Wyckoff Accumulation / Distribution weight 0.8

Idea: big players build positions inside sideways ranges. A fake-out dip below the range that recovers (a "spring") marks accumulation → up next. A fake-out above that fails (an "upthrust") marks distribution → down next.

Code: detects a tight range (<12% width) after a trend, then looks for a spring or upthrust in the last 8 bars, requiring the close to be back in the appropriate half of the range.

Group D — Quantitative / systematic

14. Mean Reversion (z-score / Bollinger extremes) weight 0.8

Idea: price stretched far from its own average tends to snap back — *unless the market is strongly trending*, when "stretched" just keeps stretching.

Code: z-score = (price – 20-bar average) / standard deviation. $|z| \geq 2$ or a close outside the Bollinger Bands fires a fade signal — but if $ADX \geq 28$ (strong trend), the signal's strength is cut to a quarter. That ADX filter is the "volatility filter" that separates this from a naive implementation.

15. Trend Following (EMA stack + Golden Cross + ADX) weight 1.2

Idea: the simplest edge in markets — things in motion tend to stay in motion. Price above a rising short average above a rising long average = ride it.

Code: bullish needs price > EMA20 > EMA50 **and** the 50-day average above the 200-day ("golden cross" regime). ADX $\geq$ 25 upgrades strength (trend has real force).

16. Breakout (Donchian channel + volume) weight 0.8

Idea: a close above the highest high of the last 20 bars signals escape from the recent range; if volume is 1.3× normal, real participation confirms it.

Code: classic Donchian channel breakout, volume-confirmed strength 0.8 vs 0.5 unconfirmed.

Group E — Options-derived

17. Options Positioning (Put/Call ratio + ATM IV) weight 0.6

Idea: the options market is a real-money opinion poll. Heavy put buying (P/C open-interest > 1.3) = bearish positioning; call-heavy (< 0.7) = bullish; an *extreme* P/C > 2.0 flips to contrarian-bullish (peak fear often marks bottoms). ATM implied volatility is reported for context.

Code: sums open interest across the nearest expiry chain from Yahoo. Silently skipped for stocks without listed options (e.g. NSE names on Yahoo).

5. How 17 Opinions Become One Flag

The aggregation in `scoring.py` is a **weighted vote**, the same shape as a linear ensemble in machine learning:

```
score =  $\sum (\text{direction} \times \text{strength} \times \text{weight}) / \sum (\text{all weights})$       → a number in [-1, +1]

flag  = BUY  if score ≥ +0.22
        SELL if score ≤ -0.22
        HOLD otherwise

confidence = 0.6 × min(1, |score|/0.5) + 0.4 × agreement
           where agreement = |bull weight - bear weight| / (bull + bear weight)
```

Worked example: daily structure up (+1 × 0.8 × 1.5 = +1.2), golden-cross trend following (+1 × 0.85 × 1.2 = +1.02), weekly RSI divergence bearish (−1 × 0.7 × 1.3 = −0.91), everything else neutral. Net +1.31 over total weight ≈ 15 → score ≈ +0.09 → **HOLD** — bullish lean, but not enough agreement to call BUY. The confidence formula rewards *one-sidedness*, not just magnitude: 20 signals split 11/9 is a coin flip even if the score clears a threshold.

The weights are currently **judgment calls** (structure and trend matter more than harmonics). The backtester exists precisely to replace judgment with evidence — its per-strategy report shows which detectors actually predicted forward returns, and weights can then be re-fit. This "hand-tuned prior → data-driven posterior" story is a strong interview talking point.

6. The AI Technology, Explained Honestly

6.1 Where the LLM sits (and where it deliberately doesn't)

Google **Gemini** (free tier, REST API) powers only the chat. Every request sends:

1. **A system prompt** — the job description: "you are a technical analyst embedded on TradingView; ground every claim in the provided analysis; cite concrete levels; be concise; always end with 'Not financial advice'."
2. **The evidence** — the engine's compacted output (flag, score, every signal with its timeframe, direction, and detail text) as JSON in a second system message.
3. **Chat history** — the last 8 turns for that symbol, so follow-ups have context.
4. **Your question.**

This is **grounding** (a small-scale form of RAG — retrieval-augmented generation): instead of letting the model answer from its training memory (stale, hallucination-prone), we hand it the authoritative facts at request time and instruct it to stay inside them. Temperature is set low (0.3) because we want faithful explanation, not creativity.

6.2 Reliability engineering around the LLM

- **Model fallback ladder:** `gemini-flash-latest` → `gemini-flash-lite-latest` → `gemini-2.0-flash`. On 503 (overloaded) or 429 (quota), try the next. Aliases like "-latest" also survive Google retiring specific versions.
- **Graceful degradation:** no API key or all models down → a rule-based formatter turns the same evidence into a fixed-format answer. The product never fully breaks when the AI is unavailable — a pattern worth naming in interviews: *the LLM is an enhancement layer, not a dependency*.
- **Cost control by design:** the evidence JSON is compacted (only name/timeframe/direction/detail per signal) to keep prompts small, and analyses are cached 5 minutes server-side.

6.3 The agents package — four LLM-agent patterns

The repo also contains a study set of agent architectures (they power ad-hoc research, and the engine is exposed to them as the `get_technical_flag` tool):

- **Tool use / function calling** (`simple_agent.py`): the LLM receives JSON schemas of 18 tools, decides which to call with what arguments, gets results back, and composes an answer — the core agent loop.
- **Conversational memory** (`conversation_agent.py`): message history + tracked entities ("it" resolves to the ticker under discussion).
- **Structured outputs** (`structured_agent.py`): the model must return a Pydantic-validated 30-field object (enums for recommendation, bounded confidence 1–10). Guarantees machine-readable output — no regex parsing of prose.
- **Local open-weights** (`free_agent.py`): the same loop hand-implemented against Ollama (Qwen3/Llama3) — shows what the SDK abstracts: the while-loop over `tool_calls`, executing functions, appending `role:"tool"` messages.

7. Engineering Concepts Used (Interview-Ready Vocabulary)

Concept	Where in this project · what to say about it
REST API design	FastAPI server with resource-style endpoints; Pydantic request models give free validation and self-documenting schemas (/docs).
Stateless worker	The scanner derives every decision from (flags now) + (positions now). No local memory → identical behavior on laptop or ephemeral CI machine; crash-safe; horizontally trivial.
Idempotency-adjacent design	Buys only fire when no position exists; sells only when one does. Re-running a scan cannot double-buy. (True idempotency keys are listed as a go-live hardening step.)
Circuit breaker	Daily drawdown $\geq 3\%$ → the scanner flips its own kill switch (AUTO_TRADING=off) and stops. Fails safe, not silent.
Caching with TTL	5-minute in-memory analysis cache: bounds API load, keeps the sidebar snappy; failures are never cached (rate-limit errors are transient).
Graceful degradation	Three independent examples: Gemini→rule-based text; Yahoo→Alpaca IEX bars; GitHub-toggle unconfigured→manual variable editing. Every dependency has a documented failure mode.
Git-as-state / GitOps	Portfolio + watchlists live in the repo; cloud runs commit changes back; local edits go through the contents API. Free, auditable (every trade is a commit), and conflict-aware.
Scheduled serverless	GitHub Actions cron with a variable gate (skipped job = \$0). Cron in UTC, two market windows, concurrency group prevents overlapping runs.
Secrets management	Keys in gitignored env files locally, encrypted repo Secrets in CI, fine-grained PAT scoped to one repo with two permissions. Code never contains a credential.
Browser extension architecture	MV3: content script (page world, UI) ↔ service worker (extension world, network) via message passing — the sanctioned way around CSP/private-network blocking of https→http://127.0.0.1 calls.
Walk-forward evaluation	The backtester only ever feeds the engine data up to the simulated "today" — the time-series equivalent of train/test discipline; prevents look-ahead bias.
Observability	Flag journal (SQLite) + outcome scoring + per-strategy attribution = the product's own evaluation harness.

8. The Accountability Layer

8.1 Flag journal & scoreboard

Every fresh flag is logged (symbol, flag, score, confidence, price, time) — deduplicated so sitting on one chart doesn't spam. After 5/10/20 *trading days*, the outcome updater fetches what price actually did and stores the return. The /performance scoreboard then reports, per flag type and horizon: sample count, average forward return, and hit rate (BUY correct if price rose, SELL if it fell). This converts "the bot feels smart" into measurable precision.

8.2 The backtester

For each symbol: start 250 bars in, step forward 5 trading days at a time, compute the flag with *only past data* (daily + weekly frames), record forward 5/10/20-day returns. Reports:

- **Per-flag:** hit-rate and average return vs the buy-and-hold baseline (a BUY signal is only valuable if it beats the market's natural drift).
- **Per-strategy attribution:** for every individual signal vote, the return *signed by the vote's direction* — a strategy with positive signed mean pointed the right way more than drift. Early 2-symbol run: supply/demand strongest (+1.98% vs +0.18% drift); mean-reversion and liquidity-sweep negative — exactly the evidence weight-tuning needs.

Honest limitations, stated in the report itself: 1W+1D signals only (no intraday/options history on free data), Yahoo adjusted prices, fills approximated at the sample bar.

8.3 The virtual broker (India)

A ~200-line simulated brokerage: cash + positions + trade log + equity snapshots in one JSON file. Whole-share fills at the latest price, ₹1,000 budget with a minimum-one-share fallback (Indian large-caps often cost more than ₹1,000/share), long-only. The /performance dashboard renders it like a broker app: performance chart (portfolio % vs NIFTY 50 %, one axis, range chips, hover tooltip), holdings with per-position P&L, ALL HOLDINGS / TOTAL P&L / CASH / TOTAL summary, and a custom-amount reset button.

9. The GitHub Actions Pipeline — Our CI/CD Workflow

9.1 What we built, in plain words

GitHub Actions is GitHub's built-in automation service: you commit a YAML file describing *when* to run and *what* to run, and GitHub executes it on a fresh Linux virtual machine in their cloud — free for 2,000 minutes/month on private repos. Classic **CI/CD** uses this to build → test → deploy code on every push. Our pipeline is the same machinery used for a different job: a **scheduled production workload** — the trading scanner — that must run when no laptop is on. (Being precise about that distinction — "I used the CI/CD platform as a serverless cron with a deployment-grade release gate" — is itself a good interview line.)

9.2 The workflow file, line by line

```
name: Paper Trader

on:
  schedule:
    - cron: "*/30 13-20 * * 1-5" # every 30 min, US market hours (UTC!)
    - cron: "*/30 3-10 * * 1-5"  # every 30 min, NSE market hours (UTC)
  workflow_dispatch: {}          # manual "Run workflow" button

permissions:
  actions: write                 # scanner can flip the AUTO_TRADING variable
  contents: write                # the run can commit the NSE portfolio back

concurrency:
  group: paper-trader           # never two scans at once
  cancel-in-progress: false

jobs:
  scan:
    if: ${vars.AUTO_TRADING == 'on' || github.event_name == 'workflow_dispatch'}
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements-scanner.txt
      - name: Run watchlist scan
        env:
          ALPACA_API_KEY:    ${secrets.ALPACA_API_KEY}
          ALPACA_SECRET_KEY: ${secrets.ALPACA_SECRET_KEY}
          GITHUB_TOKEN:      ${github.token}
        run: python run_scanner.py
      - name: Commit NSE virtual portfolio (if it traded)
        run: |
          if [ -n "$(git status --porcelain nse_portfolio.json)" ]; then
            git config user.name "paper-trader-bot"
            git config user.email "actions@users.noreply.github.com"
            git add nse_portfolio.json
            git commit -m "NSE virtual portfolio update [skip ci]"
            git pull --rebase origin main
            git push
          fi
```

Element	Why it's there (the design reasoning)
Two cron lines	One pipeline serves two markets in two time zones. Cron is always UTC: 13:30–20:00 UTC = US session; 03:45–10:00 UTC = NSE session. One workflow, one gate, one audit trail.

<code>workflow_dispatch</code>	A manual trigger for testing — it deliberately <i>bypasses</i> the AUTO_TRADING gate (see the <code>if:</code>), so you can verify the whole chain end-to-end on a weekend without enabling the schedule. This is our "deploy to staging manually" equivalent.
The <code>if:</code> gate on repo variable	The release gate: <code>vars.AUTO_TRADING == 'on'</code> . A skipped job bills ~0 minutes, so OFF costs nothing. The gate lives on GitHub's side, which is what makes the laptop-off toggle possible — and the sidebar button just edits this variable via the REST API.
<code>permissions: block</code>	Least privilege for the auto-issued <code>GITHUB_TOKEN</code> : exactly <code>actions:write</code> (drawdown breaker turns the variable off) and <code>contents:write</code> (commit the portfolio). Nothing else — no issues, no packages, no pages.
<code>concurrency group</code>	If a run overruns 30 minutes (rate-limit retries), the next scheduled run queues instead of trading in parallel — the race-condition guard for a system that moves (paper) money.
<code>timeout-minutes: 15</code>	A hung run can't eat the minutes budget or hold the concurrency lock forever.
<code>cache: pip</code>	Dependency caching drops install time from ~90s to ~15s per run — at ~340 runs/month that's real budget. Same reason we keep a minimal <code>requirements-scanner.txt</code> instead of the full dev requirements.
Secrets via <code>env:</code>	Alpaca keys are encrypted repo Secrets, injected as environment variables only into the step that needs them; they never appear in code, logs (GitHub masks them), or the checkout.
The commit-back step	The "GitOps state write": if the scanner changed <code>nse_portfolio.json</code> , commit and push it. <code>git status --porcelain</code> catches both modified <i>and</i> brand-new files (a plain <code>git diff</code> misses untracked files — a real bug we fixed). <code>git pull --rebase</code> handles a human pushing between checkout and push. <code>[skip ci]</code> in the message prevents the bot's own commit from triggering pipelines — the classic infinite-loop guard.

9.3 How the pipeline is actually used, day to day

- **The ON/OFF lifecycle:** sidebar ☰ toggle → PATCH the AUTO_TRADING variable via API (fine-grained PAT) → next cron firing sees the new value. The drawdown circuit breaker uses the identical path in reverse — the pipeline can disable itself.
- **Deployment model:** "deploying" new trading logic = `git push` to main. The next scheduled run checks out the latest commit — every run deploys fresh. Rollback = `git revert` . There is no separate build artifact, which is appropriate for interpreted Python at this scale.
- **Observability:** the Actions tab is the ops console — every run's full stdout (each symbol's flag, every decision, every guardrail trip) is a permanent, timestamped log. The bot's portfolio commits double as an audit trail: every trade is literally a commit.
- **Budget:** ~16 runs/market-day/market × 2–3 min ≈ 700–1,000 minutes/month vs the 2,000 free. Skipped (OFF) runs and holiday early-exits cost ~0–1 minute.

What a "full" CI/CD setup would add (and honest reasoning for why we haven't yet): a second workflow triggered `on: push` running lint (ruff) + unit tests on the strategy math + a smoke test (`run_scanner.py --dry-run` against recorded fixtures), required to pass before merge via branch protection. Single-developer velocity made us defer it; it is the first thing to add before real money, because at that point a bad push auto-deploys into a live trader within 30 minutes.

10. Real Money: The Complete Transition Guide

To be used only after months of paper trading. The most important parts are the reasons *not* to go live on any given day.

Phase 0 — Define "success" before looking at results

1. **Sample size:** ≥ 3 months of auto-trading AND ≥ 60 closed trades. Fewer = luck dominates.
2. **Beats the boring alternative:** portfolio return $>$ NIFTY 50 (India) and $>$ S&P 500 (US) over the same period. 4% while the index made 7% is a loss.
3. **Survivable drawdown:** worst paper peak-to-trough $< 15\%$ — and imagine it in real money; live will feel worse.
4. **Not one lucky stock:** remove the single best trade — still profitable?
5. **Backtest agreement:** the 5-year backtest credits the same strategies as the live journal. Disagreement = regime luck.

Phase 1 — Non-negotiables

1. **Employer compliance first.** Investment-industry employers almost always have personal-trading policies (pre-clearance, restricted lists, rules on automated trading). Ask before opening a live account — no other step can end a career.
2. **Capital rule:** only money whose total loss changes nothing. Not savings, never borrowed.
3. **Taxes:** every sale is a taxable event; rules differ by residency and market. One conversation with a tax professional before, not after.

Phase 2 — US route (Alpaca live)

1. Open a live Alpaca account (full KYC; international supported; wire/ACH funding).
2. Generate live keys — they start with `AK` (paper keys start `PK` ; the prefix is your safety check).
3. The code switch is two env vars (`ALPACA_BASE_URL=https://api.alpaca.markets` + live keys) — but only after the Phase 4 hardening.

Phase 3 — India route (structurally different)

- The virtual broker is a simulation and cannot go live. Real automated NSE trading needs a broker API: Zerodha Kite Connect (~₹2,000/mo), Upstox, Angel One SmartAPI — and SEBI's retail-algo framework imposes registration/approval requirements that change; verify current rules with the broker.
- **Pragmatic recommendation: keep India semi-automatic.** The bot signals; you execute in your real broker app. ~90% of the value, 0% of the regulatory/API complexity — and with ₹1,000–4,000 positions, API fees can't be justified anyway.

Phase 4 — Engineering hardening (required before live keys)

1. **Environment separation:** live keys only in GitHub Secrets + a separate env file; a `TV_BOT_LIVE=1` flag with a loud startup banner.
2. **Limit orders** instead of market orders (paper fills hide slippage; live market orders at the open can fill badly).
3. **Idempotency guard:** skip a buy if a `tvbot-` order for that symbol already exists today.
4. **Tighter breakers:** daily 3% $\rightarrow$ 2%; add weekly $-5\% \rightarrow$ off for the week; add max-consecutive-losses stop.
5. **Phone notifications** for every live order and breaker trip (Telegram bot API / ntfy.sh — free).
6. **Kill-switch drill:** prove you can flip `AUTO_TRADING` off from your phone in < 60 seconds and flatten everything in the broker dashboard. Practice once.

Phase 5 — Staged rollout

1. **Month 1, ~\$500, \$50–100 positions.** Goal is measuring live-vs-paper friction, not profit.
2. **Shadow comparison:** keep paper running the same watchlist; live minus paper = your real friction cost. If live lags >1–2%/month, the edge may not survive fees+slippage — stop.
3. **Scale by doubling only after profitable/flat months**, up to the Phase-1 cap. Never add money after a losing month to "win it back".
4. **Monthly ritual:** hit-rate vs paper history, drawdown vs gate, friction, and: "would I have made these trades myself?"
Frequent overriding = you don't trust it = back to paper.

Three closing truths: paper success is necessary, not sufficient (slippage, fees, taxes, psychology). The bot's real product may be the *analysis*, not the autopilot — running it as decision support forever is a legitimate outcome. And none of this is financial advice; it is process discipline for a decision that is entirely yours.

11. Optimization Roadmap — AI/ML Not Yet Used

The current system is deliberately deterministic. Each idea below adds genuine machine learning, ordered roughly by value-per-effort. (These are also excellent "how would you extend it?" interview answers.)

1. ML meta-model over the signals (the highest-value upgrade)

The engine already produces a perfect **feature vector**: 20–30 signals per analysis, each with direction/strength/timeframe. Train a gradient-boosted tree model (XGBoost/LightGBM) where X = those signals and y = "did price rise $\geq 2\%$ within 10 days". The engine becomes the feature factory, the model learns which *combinations* predict outcomes (e.g. "demand zone + weekly uptrend" matters; "harmonic alone" doesn't). Evaluate with **walk-forward cross-validation only** — never random k-fold on time series (leakage). Feature importance (SHAP) then gives an explainable replacement for the hand-set weights.

2. Data-driven weight optimization (the lighter version of #1)

Keep the linear vote, fit the weights: grid search / Bayesian optimization (Optuna) / genetic algorithms over backtest hit-rates, with time-series CV and a holdout period. Guard against overfitting: fewer parameters than #1, but the same leakage discipline applies.

3. Market regime detection

Strategies behave differently in trending vs choppy vs panic markets — the backtest already hints at this (mean reversion loses in trends). Fit a **Hidden Markov Model** or clustering (k-means on volatility/correlation features) to label the current regime, then switch weight sets per regime. This is the classic fix for "the backtest worked until the market changed".

4. NLP sentiment as an 18th signal

The repo already has news/social/SEC tools. Pipe headlines through **FinBERT** (finance-tuned BERT classifier, free, runs locally) or batch-score with Gemini into a $-1 \dots +1$ sentiment feature with decay weighting. Key ML lesson to mention: verify sentiment *leads* price rather than lags it — most naive news-sentiment signals are reactive, not predictive.

5. RAG over fundamentals (10-K/10-Q, earnings calls)

Chunk + embed SEC filings into a local vector store (FAISS/Chroma + sentence-transformers), retrieve relevant passages per question, let Gemini answer with citations. Upgrades the chat from "technical analyst" to "full analyst", and is the canonical RAG system-design story for interviews: chunking strategy, embedding choice, retrieval evaluation, hallucination control via citation grounding.

6. Meta-labeling (López de Prado)

Don't ask ML "which way will price go?" — keep the engine for direction, train a classifier only on "given this flag fired, will it WIN?" using signal context as features. The model's probability output then **sizes** the position (skip / small / full). Meta-labeling is respected in quant literature precisely because it layers ML on top of an explainable base signal.

7. Smarter position sizing (Kelly / volatility targeting)

Replace fixed ₹1,000/\$1,000 with fractional-Kelly sizing from journal win-rates, or volatility-targeted sizing (risk the same ATR-multiple per trade). Cheap to build, large effect on drawdown — sizing often matters more than entries.

8. Deep learning forecasters — with honest caveats

LSTM/Temporal-Fusion-Transformer models on OHLCV sequences are the obvious "add deep learning" answer, and the honest reply is that daily prices are ~95% noise: naive LSTM projects typically underperform tree models on tabular features. If used, predict *volatility* or *ranking* across stocks rather than point prices. Saying this in an interview signals maturity, not weakness.

9. Reinforcement learning for execution/allocation

Frame trading as an MDP (state = signals+positions, actions = buy/sell/hold, reward = risk-adjusted P&L) and train PPO in a backtest gym. Academically interesting; practically sample-hungry and overfit-prone on one asset's history — position it as an experiment, not a roadmap item.

10. LLM analyst committee (multi-agent debate)

Three Gemini calls with different personas (bull / bear / risk manager) each argue from the same evidence; a judge synthesizes. Multi-agent debate measurably reduces single-pass hallucination and maps directly to the agentic-AI trend interviewers ask about.

11. MLOps if #1 ships

Data versioning (DVC), experiment tracking (MLflow/W&B), scheduled retraining in the same GitHub Actions, and **drift monitoring** — alert when live hit-rate drops below the backtest confidence band. Free tiers exist for all of it.

12. Interview Preparation — Questions & Answers

Grouped the way interviews actually flow. Answers are compressed to the version you'd say aloud.

A. Project & system design

Q: Walk me through the architecture.

A: Five layers. A Chrome-extension sidebar detects the stock the user views on TradingView and calls a local FastAPI server. The server runs a deterministic 17-strategy technical engine over four timeframes of Yahoo data and aggregates a weighted vote into a BUY/SELL/HOLD flag. A Gemini LLM explains that evidence conversationally — grounded, so it never invents the decision. Execution goes to an Alpaca paper account (US) and a self-built virtual broker (India). A scheduled GitHub Actions scanner re-runs the engine over watchlists during market hours and reconciles positions — stateless, so laptop-off operation works; state lives in the broker and in git.

Q: Why is the scanner stateless, and what did that buy you?

A: Every run derives decisions from current flags plus current positions fetched from the source of truth (Alpaca / a git-tracked JSON). No local DB to sync means the identical script runs on my laptop or a throwaway CI machine, crashes lose nothing, and there's no split-brain between environments. The trade-off is re-fetching everything each run — fine at a 30-minute cadence.

Q: You store a portfolio in a git repo. Defend that.

A: Requirements were: free, durable, reachable by ephemeral cloud runs, auditable. A git-tracked JSON plus commits from CI gives all four — every trade is literally a commit with a message. Limits I'd state up front: no concurrent writers (solved with an Actions concurrency group), no queries, and it would be wrong beyond one user — at scale I'd move to Postgres and treat this as what it is, a single-user GitOps pattern.

Q: How does the off-hours toggle work end-to-end?

A: The switch is a GitHub repo *variable*, AUTO_TRADING. The workflow's job has `if: vars.AUTO_TRADING == 'on'`, so OFF skips the job at zero cost. The sidebar toggle flips the variable through the GitHub REST API using a fine-grained PAT scoped to that one repo. The drawdown circuit breaker reuses the same mechanism — the bot can turn itself off.

Q: Race conditions in your system?

A: Three handled: overlapping scheduled runs (Actions concurrency group), user resetting the portfolio locally while the cloud trades it (canonical copy is GitHub's; local server re-fetches before mutating and pushes after), and duplicate buys within a run (position check before order, plus reconcile semantics — buy only if not held). For live money I'd add true idempotency keys.

Q: What breaks first at 1,000 users?

A: Everything single-user: one shared Yahoo rate limit, one server process, portfolio-in-git, secrets on one machine. I'd move data to a licensed feed with a caching layer, state to Postgres, analysis to queued workers, auth in front of the API — and the deterministic engine itself scales fine because it's pure CPU-bound computation per symbol.

B. AI / LLM questions

Q: Where exactly is the AI, and why doesn't the LLM make the trading decision?

A: The LLM only explains and answers questions, grounded on computed evidence. Decisions need to be reproducible, backtestable, and numerically precise — LLMs are none of those: same prompt can yield different answers, you can't replay them over 5 years of history meaningfully, and they hallucinate price levels. Determinism for decisions, language models for language — using each where it's strong is the design decision I'd defend hardest.

Q: What is grounding / how do you prevent hallucination?

A: I inject the authoritative facts (the signal JSON) into the system context and instruct the model to cite only from it, with low temperature (0.3). It's RAG's "augmented generation" half — retrieval is trivial because I compute the context rather than search for it. Residual risk is the model over-summarizing, mitigated by keeping evidence compact and structured. Full elimination is impossible; the fallback rule-based answer is the zero-hallucination floor.

Q: Explain function calling and how you'd implement it without a framework.

A: You send the model JSON schemas of available functions; it responds with a structured call (name + arguments) instead of text; you execute, append the result as a tool-role message, and loop until it answers in prose. The repo's FreeAgent implements exactly that loop by hand against Ollama — a while-loop over `tool_calls`. Frameworks add schema generation, validation, and retries.

Q: What are structured outputs and why do they matter?

A: Constraining the model to emit schema-valid JSON (Pydantic model / JSON-schema enforcement) — enums for the recommendation, bounded ints for confidence. It converts an LLM from a text generator into a typed API you can build systems on; validation failures retry at the API layer instead of crashing downstream code.

Q: How did you handle LLM reliability and cost on a free tier?

A: A model fallback ladder (latest-flash → flash-lite → 2.0-flash) on 429/503; "-latest" aliases so provider retirements don't break me; compacted evidence to keep tokens small; 5-minute server-side caching; and a non-LLM fallback so availability never hits zero. Cost engineering is mostly prompt and caching discipline before it's model choice.

Q: Local models vs API models — trade-offs you actually hit?

A: Ollama/Qwen: private, free, offline — but needs 10–30GB RAM, tool-calling reliability is model-dependent, and on a corporate laptop the install itself is a policy question. Gemini API: zero install, better quality, but data leaves the machine and quotas apply. I shipped the API for chat and kept the local path as an option — and the deterministic engine needs neither.

C. ML fundamentals (asked via this project)

Q: Why walk-forward validation instead of k-fold for this data?

A: Time series has temporal dependence; random folds let the model train on the future and test on the past — look-ahead leakage that inflates results. Walk-forward mimics deployment: train/decide on data up to t , evaluate strictly after t , roll forward. My backtester does that by construction — each sample's flag sees only prior bars.

Q: Sources of bias in backtests?

A: Look-ahead (using future data — prevented structurally), survivorship (testing only stocks that still exist; my fixed watchlist has mild survivorship, I'd disclose it), overfitting (tuning weights until history looks good — why I keep a holdout and report per-strategy attribution), and cost neglect (paper fills ignore slippage/fees — why the go-live plan measures live-vs-paper friction on tiny size first).

Q: If you added an ML classifier, how would you frame the problem?

A: Meta-labeling: features = the 20–30 engine signals at flag time; label = whether the flag "won" ($\geq 2\%$ favorable move within 10 days before an adverse -2%). Gradient-boosted trees for tabular data, walk-forward CV, SHAP for explainability, and the classifier's probability sizes the trade rather than overriding direction. Class imbalance handled with class weights; success metric is precision-at-threshold and net P&L uplift over the unfiltered baseline, not accuracy.

Q: Why would accuracy be the wrong metric here?

A: Classes are imbalanced and payoffs are asymmetric — a 55%-accurate model that wins 2R and loses 1R is great; a 70%-accurate one with the reverse payoff bleeds money. The right metrics are calibrated probability + expected value per trade, and ultimately risk-adjusted portfolio return (Sharpe/max drawdown), which is exactly what the journal and paper accounts measure.

Q: How would you detect the model/strategy decaying in production?

A: The journal is a live evaluation set: rolling hit-rate per flag with confidence intervals, compared against the backtest's expectation band. Drop below band → alert, halve size or flip AUTO_TRADING off, investigate regime change. Same idea as data/concept-drift monitoring in MLOps — the label just arrives 5–20 days late.

D. Engineering & quant details

Q: Why did content-script fetches to 127.0.0.1 fail, and your fix?

A: An https page calling http://localhost trips CSP and Chrome's private-network-access protections. The sanctioned pattern: route requests through the extension's background service worker, which holds host_permissions for that origin — content script and worker talk via message passing. It also centralizes error handling.

Q: How do you avoid double-trading when the schedule fires twice?

A: A GitHub Actions concurrency group serializes runs; within a run, decisions are reconciliations (buy only if flat, sell only if held), so replaying is naturally harmless; and bot orders carry a tvbot- client-order-id prefix so they're identifiable. For real money I'd add day-scoped idempotency keys checked against open orders.

Q: Whole-share simulation — why does it matter for India?

A: NSE has no fractional shares, and many large-caps cost more than a small position budget. Ignoring that makes paper results unreproducible live. I sized ₹1,000/position with a documented min-one-share fallback, and warned that starting cash must be scaled down or cash drag makes return percentages meaningless — realism details like these are where paper results usually lie.

Q: Biggest honest limitation of the whole system?

A: The signals are all public-information technicals — any persistent edge is small and decays; free data adds noise (IEX-only volume on the fallback, Yahoo throttling); and the weights started as judgment. That's precisely why the system's core loop is measurement: journal, benchmark-relative scoreboard, per-strategy attribution, and a staged path where evidence, not enthusiasm, gates every escalation.

E. GitHub Actions / CI-CD pipeline questions

Q: Is your GitHub Actions workflow really "CI/CD"? Defend your usage.

A: Strictly, CI/CD builds, tests, and deploys on code changes; mine primarily runs a scheduled production workload on the same platform. But it shares the load-bearing pieces of a deployment pipeline: a release gate (the AUTO_TRADING variable), secrets injection, least-privilege permissions, concurrency control, and continuous deployment in the literal sense — every scheduled run checks out the latest main, so pushing to main IS deploying, and rollback is git revert. What it lacks — and I'd add before real money — is the CI half: lint + unit tests + a dry-run smoke test gating merges, because auto-deploying an untested push into a trader within 30 minutes is unacceptable at live risk.

Q: Why gate with a job-level `if:` on a repo variable instead of a step that exits early?

A: Three reasons. Cost: a skipped job never provisions a runner, so OFF is ~0 billable minutes vs ~1 minute per early-exit run — at ~680 potential firings a month that's the difference between 0 and ~700 wasted minutes. Control-plane placement: the variable lives on GitHub, so it's flippable from any device via API when the laptop is off. And auditability: skipped runs are visibly "skipped" in the Actions tab rather than a wall of no-op successes.

Q: `GITHUB_TOKEN` vs a personal access token — when did you need each?

A: Inside the workflow, the auto-issued GITHUB_TOKEN suffices — scoped to the repo, short-lived, with permissions I narrowed to actions:write + contents:write in the permissions block. Outside the workflow — my local server flipping the toggle or syncing watchlists — there is no ambient token, so I created a fine-grained PAT restricted to that single repo with exactly Variables and Contents read/write. Least privilege on both paths; neither token can touch other repos.

Q: Why `[skip ci]` in the bot's commit message?

A: The workflow commits the portfolio file back to the branch that (in a fuller setup) triggers workflows on push — a self-triggering loop. `[skip ci]` tells GitHub not to start pipelines for that commit. Belt-and-suspenders: pushes made with the default GITHUB_TOKEN don't trigger workflows anyway, precisely to prevent recursive Actions — knowing both layers is the senior-level answer.

Q: What does your concurrency group protect against, and why cancel-in-progress: false?

A: Two scans trading simultaneously — e.g. a rate-limited run overrunning into the next cron slot — could each see "no position" and double-buy. The group serializes them. cancel-in-progress is false because killing a half-finished trading run mid-order is worse than queueing the next one; for a docs-site deploy I'd choose true (only the latest matters).

Q: Gotchas with `schedule`: triggers you actually hit or designed for?

A: Cron is UTC only — both market windows are converted (13:30–20:00 UTC for New York, 03:45–10:00 UTC for NSE) and I accept DST drift or adjust seasonally. Scheduled runs can start minutes late under load — fine for daily-timeframe signals, disqualifying for HFT. GitHub auto-disables schedules on repos with 60 days of no activity — my bot's own portfolio commits keep it active. And scheduled workflows only run from the default branch — feature-branch schedules silently don't fire.

Q: How do secrets flow through the pipeline, and what prevents leaking them?

A: Stored as encrypted repo Secrets, injected as env vars only into the single step that needs them, never written to the checkout. GitHub masks secret values in logs automatically, but I also don't print them and the scanner treats them as opaque. The PDF-worthy detail: secrets are not passed to workflows triggered from forks — irrelevant for a private solo repo, critical to mention for OSS.

Q: A run fails at 2 AM. Walk me through your debugging.

A: Actions tab → the red run → expand the failing step; the scanner prints per-symbol decisions and every guardrail trip, so the log usually names the cause directly (Yahoo 429 → the Alpaca-bars fallback should have caught it; Alpaca 4xx → key or symbol issue; git push rejection → a concurrent human push, handled by pull --rebase). Because the scanner is stateless and reconciling, the standard remediation is simply "let the next run happen" — no state to repair, which was the point of the design.

Q: How would you extend this into a full CI/CD pipeline for the project?

A: A second workflow on pull_request: ruff lint, mypy, pytest over the indicator/structure math with golden-file fixtures, and run_scanner.py --dry-run against recorded API responses (vcr.py) as a smoke test; branch protection requiring it green before merge to main. Then the existing scheduled workflow becomes the CD leg deploying only reviewed code. For live trading I'd add an environment with a required reviewer on the live-keys environment — GitHub's built-in manual-approval gate.
